

PRISM-Loc: Three LiDAR Localization Backends Behind One ROS 2 Contract, with Middleware-Free Estimator Cores

Jung Mo Kang
kangjmo91@gmail.com

Abstract—Map-based LiDAR localization for mobile robots is solved in fragments: a 2D scanner calls for an AMCL-style particle filter over an occupancy grid, a 3D LiDAR for an NDT-based tracker over a point-cloud map, and an inertial- or GNSS-aided platform for a Kalman fusion node—each with its own topics, parameters, initialization ritual, and recovery behavior. We present `prism_loc`, an open-source ROS 2 localization stack that places three known estimator families behind one node contract: `laser2d`, a likelihood-field Monte Carlo localizer; `ndt3d`, an NDT-MCL backend that reuses the same particle filter with a Gaussian-voxel measurement model; and `fusion3d`, a 15-state error-state Kalman filter fusing LiDAR, IMU, and RTK-GNSS. Every backend publishes the same map→odom transform and pose with covariance and accepts the same `/initialpose` seeding, so the sensor suite selects a backend at launch time rather than a separate software stack. All filter mathematics reside in middleware-free C++17 cores that depend only on Eigen and are exercised by deterministic, seeded unit and integration tests with no ROS runtime; the `laser2d` backend additionally recovers from the kidnapped-robot condition by deterministic branch-and-bound scan matching. We claim no new estimator theory: the contribution is architectural, delivered as a usability-focused open-source release.

I. Introduction

Localizing a mobile robot against a prior map is, on paper, a solved problem, but in practice it is solved in fragments. A team fielding a 2D LiDAR reaches for `nav2_amcl`, a particle filter that tracks a planar pose against an occupancy grid and nothing else. A team with a 3D LiDAR reaches instead for an `hdl_localization`-style tracker built around 3D NDT registration. A team that must fuse inertial and satellite measurements reaches for a `robot_localization`-style node [1], which composes an EKF over odometry, IMU, and GNSS but treats the map-relative LiDAR measurement as someone else’s problem. Each of these tools carries its own topic interface, parameter vocabulary, initialization ritual, and recovery story, so a practitioner who moves from a 2D platform to a 3D one, or who adds a GNSS receiver, re-integrates from scratch. The ROS 2 maintainers’ localization survey [2] names exactly this gap and sketches, as unbuilt future work, a modular framework of swappable 2D and 3D localization components behind a common interface (Section II details this and the surrounding prior work). The cost of the gap is not merely inconvenience: the Nav2 marathon deployment [3] attributes a leading share of its autonomy-

recovery triggers to loss of localization confidence, so how a stack initializes, degrades, and recovers is a first-class deployment concern rather than a detail.

`prism_loc` is an engineering response to that gap. It packages three map-based LiDAR localization backends behind one ROS 2 node contract: `laser2d`, a 2D likelihood-field Monte Carlo localizer against an occupancy grid; `ndt3d`, a 3D NDT-MCL backend that reuses the same particle filter with a voxel-Gaussian measurement model over a prior point cloud, sampling the planar (x, y, θ) pose; and `fusion3d`, a 15-state error-state Kalman filter (ESKF) that fuses 3D LiDAR, IMU, and RTK-GNSS with IMU-bias estimation. All three present the identical interface — a map→odom transform, a pose with 6×6 covariance, and `/initialpose` seeding — so that choosing between a 2D scan, a 3D cloud, and an inertial/GNSS-aided configuration is a launch-time decision rather than a change of stack. The two MCL backends share a single pure-C++17 estimator core (motion model, particle filter, KLD-adaptive resampling, pose estimate), and the fusion backend is a second such core; in both cases the algorithmic code depends only on Eigen and is unit-tested with no ROS or PCL present. When `laser2d` is started with no initial pose, it recovers one from a single scan by deterministic branch-and-bound (BBS) correlative scan matching over the occupancy grid, and re-seeds the particle filter from that result. We claim no new estimator theory: likelihood-field MCL, NDT-MCL, ESKF fusion, and BBS scan matching are all established. The contribution is architectural — three known estimator families placed behind one contract, factored so their cores are deterministically testable off-robot — and it is delivered as usable open-source software.

Concretely, this paper contributes:

- 1) A unified, pluggable localization architecture in which three estimator families (2D likelihood-field MCL, 3D NDT-MCL, and LiDAR+IMU+RTK-GNSS ESKF fusion) present one ROS 2 map→odom contract, so the sensor suite selects a backend rather than a separate stack.
- 2) ROS-free estimator cores that hold all filter mathematics behind an Eigen-only, PCL-free, `roscpp`-free boundary, making every algorithmic claim exercisable by deterministic, seeded-RNG unit and

integration tests that run without a middleware runtime.

- 3) A deterministic global relocalization path for laser2d: branch-and-bound correlative scan matching solves the kidnapped-robot problem from one scan and re-seeds the MCL posterior, replacing stochastic random-injection recovery with a repeatable search.
- 4) An open-source release engineered for usability, including fail-fast configuration validation, startup watchdog diagnostics that name the input that has gone silent, and a single-source parameter reference kept next to the code that reads it.

Section II situates `prism_loc` against prior localization and fusion work. Section III describes the shared contract and the core/ROS boundary; Section IV details the estimator cores and Section V the ROS 2 nodes. Section VI presents the ROS-free testing and CI strategy that stands in for on-robot benchmarks in this software-description paper, together with a seeded synthetic evaluation that demonstrates closed-loop tracking, global relocalization, and fusion behavior of the actual cores, and Sections VII and VIII discuss limitations and conclude.

II. Related Work

Monte Carlo localization. Particle-filter localization was introduced by Dellaert et al. [4] and Fox et al. [5], who represent the pose posterior as a weighted sample set updated by motion-model prediction, sensor-likelihood weighting, and resampling, and who demonstrated global localization from a uniform prior on real robots. Thrun et al. [6] later hardened MCL against accurate sensors and the kidnapped-robot problem via Mixture-MCL’s dual sampler. The particle-filter core in `prism_loc` is a direct descendant of this line of work and of its `nav2_amcl` lineage; we contribute no new filter theory. What differs is factoring: the same predict-weight-resample core is shared, unchanged, between a 2D likelihood-field backend and a 3D NDT backend, and recovery from localization failure is delegated to a deterministic branch-and-bound search (Section IV-B) rather than the stochastic random-injection or dual-sampling mechanisms these papers employ.

NDT representations and NDT-based measurement models. Biber and Straßer [7] introduced the Normal Distributions Transform, modeling a scan as per-cell Gaussians and reducing registration to Newton optimization of a smooth score; Magnusson et al. [8] generalized the representation to 3D voxel grids for field deployment on mining vehicles. Both use NDT for deterministic scan-to-scan or scan-to-map registration that requires a reasonable initial guess. `prism_loc`’s `ndt3d` backend adopts the standard Gaussian-voxel representation but repurposes the NDT score as a per-particle likelihood inside the shared MCL core (i.e., NDT-MCL rather than NDT registration), which trades the quadratic convergence of Newton-based

alignment for the multi-hypothesis robustness of the particle filter, and lets the 2D and 3D backends differ only in their measurement model.

LiDAR-inertial and GNSS fusion. Solà’s tutorial [9] is the mathematical basis of `prism_loc`’s fusion backend: the nominal/error-state decomposition, local angular-error parameterization, and predict-correct-inject-reset cycle are implemented essentially as derived there. On the systems side, Moore and Stouch’s `robot_localization` [1] established the pattern of a generalized, configurable EKF/UKF fusion node for ROS, and Wan et al. [10] demonstrated an industrial ESKF fusing GNSS-RTK, LiDAR, and IMU at centimeter accuracy for the Baidu Apollo fleet, albeit as a closed system tied to proprietary intensity/altitude maps. Tightly-coupled LiDAR-inertial odometry such as FAST-LIO [11], and applied 3D map-based tracking systems such as Koide et al. [12], are complementary: they produce ego-motion and maps, whereas `prism_loc` localizes within a prior map. Relative to these, our `fusion3d` backend claims no estimator novelty; it packages a textbook ESKF as a ROS-free core behind the same ROS 2 contract as the two MCL backends, so that odometry-only, LiDAR-only, and GNSS-aided configurations are deployment choices rather than separate stacks.

Global (kidnapped-robot) localization. Olson [13] formulated 2D scan matching as exhaustive correlative search over a rasterized log-likelihood table with coarse-to-fine multi-resolution pruning, and Hess et al. [14] bounded that search with branch-and-bound over a precomputed grid pyramid to run loop closure in real time inside Cartographer. `prism_loc` borrows exactly this branch-and-bound-over-likelihood-pyramid primitive but applies it in a narrower setting: one-shot relocalization against a known static map, with no submaps or pose graph to maintain. This replaces the heuristic recovery mechanisms of classical MCL [5], [6] with a deterministic search whose result re-seeds the particle filter.

Robot software systems. Nav2 [3] treats localization as an external, swappable component (stock AMCL or SLAM Toolbox), and its marathon deployment attributes a leading share of recovery triggers to localization-confidence loss in feature-poor corridors. The ROS 2 maintainers’ survey [2] makes the gap explicit: it catalogs AMCL’s shortcomings and sketches, as unbuilt future work, a modular framework for composing 2D and 3D localization systems from swappable components. As framed in Section I, `prism_loc` answers with engineering rather than a new algorithm: three known estimator families (likelihood-field MCL, NDT-MCL, ESKF fusion) are placed behind one ROS 2 topic and TF contract, with each estimator implemented as a ROS-free library that is unit-testable without a middleware runtime — a property that `robot_localization`’s otherwise modular design [1] did not prioritize. The accuracy evaluation planned in Section VII will follow the trajectory-error conventions (ATE/RPE)

standardized by the TUM RGB-D benchmark [15], applied to per-backend map→odom estimates rather than visual SLAM output.

III. System Architecture

`prism_loc` is organized as two layers realized by four packages (Fig. 1). The lower layer holds the estimator cores: `prism_loc_core`, which implements the Monte Carlo localization (MCL) family, and `prism_loc_fusion`, which implements the error-state Kalman filter (ESKF). Both are plain C++17 libraries whose only third-party dependency is Eigen; each CMakeLists.txt calls `find_package(Eigen3 REQUIRED)` and links `Eigen3::Eigen` alone, with no reference to `roscpp`, `tf2`, or `PCL`. The upper layer holds the ROS 2 node packages: `prism_loc` (the `laser2d` and `ndt3d` backends) and `prism_loc_fusion_ros` (the `fusion3d` backend). These packages, and only these, depend on `roscpp`, the `tf2` stack, the ROS message packages, and `PCL`. Each core is consumed by its node package through the ordinary `ament find_package` mechanism, so the dependency direction is strictly upward: the cores never link against the middleware.

Why the split. Isolating the estimators from the middleware buys three properties. First, unit-testability without a running graph: each core builds and tests with a plain system toolchain and `gtest`, so its numerical behavior is exercised with no ROS installation, no node, and no message plumbing. The CMakeLists.txt of each core wires tests through plain `CTest` with system `gtest`, enabled by default via `BUILD_TESTING`, and runnable both inside a `colcon` workspace and standalone with a bare `cmake+ctest` invocation; continuous integration exercises this ROS-free path directly. The detailed testing story is deferred to Section VI. Second, determinism: all sampling routes through a single seeded `std::mt19937_64` wrapper (default seed 42), so a motion-prediction, weighting, or resampling step is reproducible bit for bit across runs, which is what makes the core tests assertable rather than merely statistical. Third, one output contract shared by all three backends. Regardless of backend, a node publishes the map→odom transform on `/tf`, a `PoseWithCovarianceStamped` on `~/pose`, and accepts a `PoseWithCovarianceStamped` on `/initialpose` for seeding; the MCL nodes additionally publish a particle cloud while the fusion node publishes an `Odometry` message carrying pose and velocity. Because the contract is fixed, choosing between the likelihood-field MCL, NDT-MCL, and ESKF estimators is a runtime decision: `prism_loc` reads a backend string parameter (“`laser2d`” by default) and branches on it when constructing its measurement model and when applying the correction step, and `prism_loc_fusion_ros` supplies the `fusion3d` estimator behind the identical topic and TF interface. As stated in Section II, this factoring — three known estimator families behind one ROS 2 contract — is

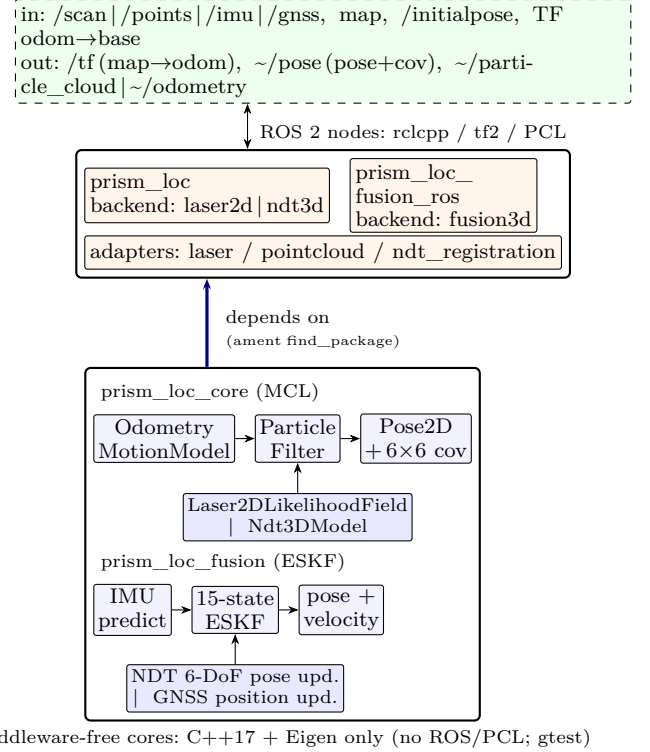


Fig. 1. `prism_loc` architecture. Two middleware-free estimator cores (bottom, C++17 + Eigen only): `prism_loc_core` runs the MCL loop (`OdometryMotionModel`→`ParticleFilter`, corrected by either the `Laser2DLikelihoodField` or `Ndt3DModel` measurement model, plus a branch-and-bound relocalizer), and `prism_loc_fusion` runs a 15-state ESKF (IMU predict, corrected by NDT 6-DoF pose and GNSS position updates). The ROS 2 node packages (top) consume the cores upward through `ament`, select a backend at runtime, bridge messages via adapters, and expose one shared contract: a map→odom TF, a pose-with-covariance output, and an `/initialpose` seed input.

the architectural contribution; the estimators themselves are textbook.

Core interfaces. The MCL core expresses the shared predict–weight–resample loop against a small set of value types. A `Pose2D` carries the planar pose (x, y, θ) ; a `Particle` pairs a pose with a weight; and the two backends differ only through a `MeasurementModel` abstraction whose single virtual method returns the log-likelihood of an observation given a candidate pose. `Laser2DLikelihoodField` (a 2D scan against an occupancy-grid likelihood field) and `Ndt3DModel` (a 3D cloud against a Gaussian-voxel NDT map) are its two implementations, and the same `ParticleFilter` and `OdometryMotionModel` drive both. The fusion core exposes an `Eskf` class over a 15-dimensional error state with three entry points — an IMU-driven predict, a 6-DoF updatePose fed by NDT registration, and a 3-DoF updatePosition fed by an absolute position fix — alongside a geodetic helper that converts WGS84 latitude/longitude/ altitude to a local ENU frame. Section IV details these estimators.

Adapters. The node layer bridges ROS message types

to the middleware-free core types through thin adapters, keeping the cores ignorant of message definitions. In `prism_loc`, a laser adapter converts a `LaserScan` to the core's `LaserScan2D` (honoring range overrides) and an `OccupancyGrid` to the core grid map, and a pointcloud adapter converts a `PointCloud2`, and a `.pcd` map file, into the `std::vector<Eigen::Vector3d>` the NDT map ingests; both also convert quaternion-bearing `Transform` and `Pose` messages to `Pose2D` by extracting yaw. In `prism_loc_fusion_ros`, an `ndt_registration` adapter wraps PCL's NDT to align an incoming cloud against the prior-map cloud and returns the resulting `Isometry3d` pose, which becomes the ESKF's 6-DoF measurement. PCL therefore appears only inside these adapters, never in the cores. The node-layer responsibilities — TF lookups, topic wiring, and runtime diagnostics — are described in Section V.

IV. Estimator Cores

This section describes the three estimator families that `prism_loc` places behind a single ROS 2 contract. All three are implemented as ROS-free C++ libraries: the two Monte Carlo localization (MCL) backends and the global relocalizer live in `prism_loc_core`, and the fusion filter lives in `prism_loc_fusion`. Consistent with the architectural framing of Section III, the contribution here is factoring, not new estimation theory: we reuse textbook constructions and report only what the code implements.

A. Shared Monte Carlo Localization Core

The two 2D and 3D pose-tracking backends share one particle filter. Following Thrun-style MCL, the pose posterior at time t is a weighted sample set $\{(x_t^{[i]}, w_t^{[i]})\}_{i=1}^N$ over the planar pose $x_t = (x, y, \theta)$ (Pose2D), updated by the standard predict–weight–resample recursion. Only the measurement model differs between backends, so the entire filter is written against an abstract interface rather than any specific sensor.

a) Motion model: `OdometryMotionModel` implements the sampling-based odometry model. Each control u_t is decomposed into a first rotation, a translation, and a second rotation $(\delta_{r1}, \delta_t, \delta_{r2})$ from consecutive odometry poses, and each particle is advanced by the noise-corrupted increment

$$\begin{aligned}\hat{\delta}_{r1} &= \delta_{r1} - \varepsilon(\alpha_1 \delta_{r1}^2 + \alpha_2 \delta_t^2), \\ \hat{\delta}_t &= \delta_t - \varepsilon(\alpha_3 \delta_t^2 + \alpha_4 (\delta_{r1}^2 + \delta_{r2}^2)), \\ \hat{\delta}_{r2} &= \delta_{r2} - \varepsilon(\alpha_1 \delta_{r2}^2 + \alpha_2 \delta_t^2),\end{aligned}\quad (1)$$

where $\varepsilon(\sigma^2)$ draws zero-mean Gaussian noise with the given variance and α_{1-4} (MotionParams) are the usual rotation/translation noise coefficients. A pure-rotation guard sets $\delta_{r1} = 0$ when the translation is below 10^{-3} m to avoid an ill-defined heading.

b) Particle filter: `ParticleFilter` weights each particle by the measurement model's log-likelihood, subtracting the per-step maximum before exponentiation for numerical stability, then normalizing. Resampling is triggered only when the effective sample size $N_{\text{eff}} = (\sum_i w^{[i]})^2 / \sum_i (w^{[i]})^2$ drops below `resample_threshold` times N , and is KLD-adaptive [16]: samples are drawn proportional to weight until the count reaches the Kullback–Leibler bound

$$M_x = \frac{k-1}{2\varepsilon_{\text{kld}}} \left(1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta} \right)^3, \quad (2)$$

where k is the number of occupied histogram bins seen so far. Bins are formed by discretizing (x, y) at `kld_bin_xy` and θ at `kld_bin_yaw`; the sample count is clamped to `[min_particles, max_particles]`, and $z_{1-\delta}$ (`kld_z`) and ε_{kld} (`kld_err`) are the standard-normal quantile and error bound. The pose estimate is the weighted mean of (x, y) and the circular mean of θ ; the reported 6×6 covariance carries the planar terms and marks z , roll, and pitch as unobserved.

c) Measurement models: A backend supplies a `MeasurementModel`, whose sole method returns the unnormalized log-likelihood of the current observation given a candidate pose. Two are provided. `Laser2DLikelihoodField` is a Thrun-style likelihood-field model for 2D scans [6]: an occupancy grid is preprocessed into a `LikelihoodField` that stores, per cell, the Euclidean distance d to the nearest occupied cell (computed once by a separable distance transform), and each subsampled beam endpoint contributes $\log(z_{\text{hit}} e^{-d^2/2\sigma_{\text{hit}}^2} + z_{\text{rand}}/z_{\text{max}})$, with `z_hit`, `z_rand`, `sigma_hit`, and `max_beams` as parameters. `Ndt3DModel` reuses the identical filter but scores a 3D point cloud against a Normal Distributions Transform map [7], [8]. The prior map is voxelized into per-voxel Gaussians (μ_v, Σ_v) (`NdtMap`, with small eigenvalues inflated to a fraction of the largest for numerical conditioning), and the per-particle likelihood accumulates $\log(s(p) + s_{\text{floor}})$ over cloud points p , where $s(p) = \exp(-\frac{1}{2}(p - \mu_v)^\top \Sigma_v^{-1}(p - \mu_v))$ and empty voxels score zero. The planar particle state (x, y, θ) is lifted to a 3D transform at a fixed base height, so the 3D backend estimates the same (x, y, θ) as the 2D one. This is the NDT-as-measurement-model (NDT-MCL) idea: the NDT score usually optimized by Newton registration is instead read out as a particle likelihood, so the two backends differ only in this class.

B. Branch-and-Bound Global Localization

Recovery from a lost or kidnapped estimate is handled by a deterministic correlative scan matcher, `BranchAndBoundMatcher`, following Olson's multi-resolution correlative search [13] and the branch-and-bound formulation of Hess et al. [14]. At construction the likelihood field is converted into a probability grid $p = e^{-d^2/2\sigma_{\text{hit}}^2}$, and a max-pooling pyramid of `max_depth+1` levels is precomputed, where level h stores the maximum over a $2^h \times 2^h$

cell block. A search then proceeds over discretized poses: translations within `linear_window` and rotations spanning `angular_window` at `angular_step` resolution. Coarse nodes are expanded depth-first with children visited in decreasing order of their pyramid-derived upper bound; because the max-pooled bound is admissible, a subtree is pruned only when its bound cannot exceed the best exact (leaf) score found so far, so the returned pose is globally optimal over the discretization. The result is accepted only when its score reaches `min_score_fraction` of the number of beams used; an accepted pose re-seeds the particle filter, whereas a rejected one leaves the current estimate untouched. Unlike the stochastic random-injection recovery of classical MCL, this search is deterministic and therefore directly reproducible in a unit test.

C. Error-State Kalman Filter Fusion

The fusion backend, `Eskf`, is a 15-state error-state Kalman filter that follows Solà’s quaternion formulation [9]. The nominal state (`NominalState`) carries position p , velocity v , orientation quaternion q , accelerometer bias b_a , and gyroscope bias b_g ; the error state $\delta x \in \mathbb{R}^{15}$ collects $(\delta p, \delta v, \delta \theta, \delta b_a, \delta b_g)$ with a local angular error $\delta \theta$. On each IMU sample the nominal state is integrated in closed form,

$$p \leftarrow p + v \Delta t + \frac{1}{2} a_w \Delta t^2, \quad v \leftarrow v + a_w \Delta t, \quad q \leftarrow q \otimes \text{Exp}(\omega \Delta t), \quad (3)$$

where $a_w = R(q)(a_m - b_a) + g$ and $\omega = \omega_m - b_g$ are the bias-corrected, gravity-compensated measurements, and $\text{Exp}(\cdot)$ is the $SO(3)$ exponential (so3.hpp). The error covariance is propagated with the discrete transition F and additive process noise Q built from the IMU noise parameters,

$$P \leftarrow F P F^\top + Q. \quad (4)$$

Two measurement updates share the standard Kalman gain $K = P H^\top (H P H^\top + R)^{-1}$, $\delta x = K y$, $P \leftarrow (I - K H) P$: `updatePose` applies a 6-DoF NDT pose observation whose residual stacks the position error and the local orientation error $\log(q^{-1} q_{\text{meas}})$, while `updatePosition` applies a 3-DoF GNSS position observation. GNSS fixes are mapped from geodetic coordinates into a local ENU frame through a WGS-84 LLA→ECEF→ENU conversion about a fixed datum (`GeodeticConverter`). After each update the estimated error is injected into the nominal state — additively for the vector components and multiplicatively, $q \leftarrow q \otimes \text{Exp}(\delta \theta)$, for the orientation — and the error state is reset to zero. Because the filter takes plain vectors and quaternions rather than ROS messages, the entire predict–correct–inject–reset cycle is exercised in ROS-free unit tests.

V. ROS 2 Integration and Interface Contract

The estimator cores of Section IV carry no middleware dependency; they are made deployable by thin ROS 2 node packages (`prism_loc` for the laser2d and `ndt3d` MCL backends, `prism_loc_fusion_ros` for the fusion3d

ESKF backend) whose job is to translate messages, TF, and parameters into core calls and back. We treat this boundary as a first-class interface rather than glue code, following the conventions that `robot_localization` [1] and `nav2_amcl` within Nav2 [3] established for drop-in localization: a REP-105 frame contract, a `map→odom` TF output, and `/initialpose` seeding. A robot already configured for AMCL can therefore switch backends by changing a launch file, not its TF tree or its consumers.

A. Frames and `map→odom` composition

All three backends honor the REP-105 frame chain `map→odom→base_link`, exposed as the `global_frame`, `odom_frame`, and `base_frame` parameters. Each backend estimates the robot body in the map, $T_{\text{map},\text{base}}$, but—so as not to fight the odometry source that owns `odom→base_link`—it broadcasts only the correction transform $T_{\text{map},\text{odom}}$. Writing $\oplus$ for pose composition and $(\cdot)^{-1}$ for inversion, the node computes

$$T_{\text{map},\text{odom}} = T_{\text{map},\text{base}} \oplus T_{\text{odom},\text{base}}^{-1}, \quad (5)$$

where $T_{\text{odom},\text{base}}$ is read from TF at the measurement stamp by both nodes; the fusion node, which publishes at IMU rate, falls back to the latest buffered transform—without blocking—when the odometry publisher has not yet caught up to the stamp. Equation (5) guarantees the round-trip identity $T_{\text{map},\text{odom}} \oplus T_{\text{odom},\text{base}} = T_{\text{map},\text{base}}$, so downstream consumers recover the intended body pose; both the planar core (compose/inverse on $SE(2)$) and the fusion node (`map_base.odom_base-1` on $SE(3)$ Isometries) implement it, and each is checked by a ROS-free unit test asserting the round trip to 10^{-9} . If `odom→base_link` is momentarily unavailable, the fusion node degrades gracefully to publishing `map→base_link` directly rather than emitting a wrong transform. The broadcast stamp is future-dated by `transform_tolerance` (default 0.1 s) to bound extrapolation on the consumer side.

B. Topic contract and QoS

Table I lists the per-backend contract. Inputs and outputs are uniform across backends wherever the sensing allows: every backend outputs the same `map→odom` TF and a `PoseWithCovarianceStamped` on `~/pose`, and every backend accepts `/initialpose`. QoS profiles match each publisher’s expected semantics rather than using defaults: sensor streams (`LaserScan`, `PointCloud2`, `Imu`, `NavSatFix`) are subscribed with `SensorDataQoS` (best-effort, volatile), while the occupancy-grid map is subscribed `transient_local` and `reliable` so that the single latched grid published once by `map_server` is delivered even though the subscriber joins later. A mismatch here is a common and silent failure mode for new ROS users; fixing the profiles at the interface removes it.

TABLE I

Per-backend topic contract. All backends additionally subscribe `/initialpose` and the TF `odom→base_link`, and broadcast `map→odom`.

Backend	Inputs	Outputs
laser2d	<code>/scan</code> , <code>/map</code> (OccupancyGrid)	<code>~/pose</code> , <code>~/particle_cloud</code>
ndt3d	<code>/points</code> , <code>.pcd</code> (file)	<code>~/pose</code> , <code>~/particle_cloud</code>
fusion3d	<code>/points</code> , <code>/imu</code> , <code>/gnss</code> (NavSatFix), <code>.pcd</code>	<code>~/pose</code> , <code>~/odometry</code>

C. Initialization paths

The interface offers one manual and one automatic seeding path per backend. Manually, all backends accept a `PoseWithCovarianceStamped` on `/initialpose` (the RViz “2D Pose Estimate” tool); the MCL backends spread a Gaussian particle cloud at the supplied mean and covariance, and the fusion backend loads the pose as its initial nominal state. Automatically, the two backends differ because their sensing differs. The laser2d backend can self-recover with no prior pose by running the branch-and-bound search of Section IV-B (`try_global_localization`, or the on-demand `~/global_localization` service), which re-seeds the filter from a single scan. The service flags a request that is served on the next scan; when `try_global_localization` is false (the default) the matcher is constructed on demand at the first service call, so the service works regardless of the parameter, which then only controls automatic relocation before the first pose. The fusion3d backend auto-initializes once it has both an attitude estimate—roll and pitch recovered from the IMU specific-force vector, heading from initial `_yaw`—and a position fix from the first accepted GNSS message, converted from WGS-84 to a local ENU frame; `/initialpose` overrides this when GNSS is unavailable.

D. Parameter surface

Every tunable is a declared ROS 2 parameter with an in-code default, documented in a single source of truth (`PARAMS.md`) maintained one-to-one against the `declare_parameter` calls. The surface deliberately exposes the core’s control knobs rather than hiding them: the KLD-sampling bounds (`kld_err`, `kld_z`, and the histogram bin sizes), the likelihood-field and laser-range parameters (`z_hit`, `sigma_hit`, `laser_min_range`, `laser_max_range`), the branch-and-bound search window and depth (`bbs_linear_window`, `bbs_angular_step`, `bbs_max_depth`, `bbs_min_score_fraction`), and the ESKF process noise and GNSS gates (`sigma_acc`, `gnss_min_status`, `gnss_max_pos_cov`). Because the cores are configured purely through plain structs, each parameter maps to exactly one field, and the shipped YAML files override only the subset a deployment typically changes.

E. Runtime diagnostics

A localization node that silently does nothing is the dominant first-run frustration; this release invests in making every stalled precondition observable and self-explaining. Four mechanisms cooperate. (i) Fail-fast on a missing mandatory map: the `ndt3d` and `fusion3d` nodes abort construction with an actionable `std::runtime_error` if `map_pcd_path` is empty or fails to load, rather than idling forever. (ii) Throttled waiting-state warnings: whenever a callback cannot yet act—no map, no initial pose, no filter initialization—it emits a rate-limited `WARN` that names both the unmet precondition and the remedy (e.g. “waiting for an initial pose—publish to `/initialpose` ...or enable `try_global_localization`”). (iii) A 10 s startup watchdog reports, for each mandatory input that has produced no message, the topic name and its live publisher count (via `count_publishers`), disambiguating “no publisher” from “publisher present but QoS-incompatible”; it cancels itself once all inputs are seen. (iv) Quantified rejection logging: the GNSS gate logs each rejected fix with the measured value against the threshold it violated—status below `gnss_min_status`, NaN coordinates, or position variance above `gnss_max_pos_cov`—and the NDT update likewise reports non-convergence and fitness versus `ndt_max_fitness`. These diagnostics carry no algorithmic weight; they exist so that a misconfiguration is diagnosable from the console log alone, consistent with the operational transparency Nav2 found necessary for long-term autonomy [3].

VI. Validation: Correctness Testing and Synthetic Evaluation

`prism_loc` is validated at the level of correctness, not field accuracy, in two complementary ways. First, a deterministic unit- and integration-test suite pins each estimator component to its specified behavior (Section VI-A). Second, a set of closed-loop synthetic experiments drives the same estimator cores through end-to-end localization scenarios and reports quantitative behavior (Section VI-B). Both run with no middleware, no simulator, and no recorded bags, because every estimator is a ROS-free library (Section IV).

A. Correctness Testing

The test suite comprises 44 GoogleTest cases across four packages: 25 in the shared `prism_loc_core` (the two MCL backends and global recovery), 13 in `prism_loc_fusion` (the ESKF), 4 in the `prism_loc` ROS node’s adapter layer, and 2 in `prism_loc_fusion_ros`.

What the cores are pinned to. The core tests target the components most likely to regress silently. The odometry motion model is checked both deterministically (zero-command and pure-translation cases) and statistically (the mean of noisy samples approximates the commanded

motion). Measurement models are checked for the monotonicity property they must satisfy to be usable as a likelihood: the ground-truth pose must out-score a displaced one, for both the 2D likelihood field and the 3D NDT voxel score. The particle-filter mechanics—low-variance resampling concentrating on a heavy particle, and the weighted-mean estimate—are tested directly, as are the occupancy-grid and NDT-map primitives (world/map round-trips, out-of-bounds handling, likelihood-field distances, and voxel-mean recovery). Global recovery is exercised by having branch-and-bound recover the correct global pose from a single scan—the kidnapped-robot scenario—and reject an empty scan. Two integration tests close the loop end to end, driving the full predict–weight–resample cycle until the estimate converges to ground truth for the laser2d and ndt3d backends respectively.

Fusion and math. The ESKF is pinned for both convergence and consistency: a static IMU leaves the state at rest, constant acceleration reproduces the expected kinematics, and position and pose updates pull the state toward the measurement; an integration test tracks a moving target from a deliberately wrong initial guess. Supporting identities are verified independently— $SO(3)$ exponential/logarithm round-trips and the skew/cross-product relation, geodetic ENU round-trips against a datum, and the map→odom TF composition arithmetic used to publish the correction transform.

Determinism. Every stochastic path draws from a single seedable generator exposed as the ROS parameter `seed` (default 42). The RNG tests assert that equal seeds reproduce identical sequences and that distinct seeds differ, so the statistical assertions above are reproducible run to run rather than flaky.

Continuous integration. Two CI jobs guard the contract from both sides. A core job builds `prism_loc_core` and `prism_loc_fusion` with plain CMake and runs `cctest`, requiring only a compiler and Eigen—no ROS—which keeps the cores honestly middleware-independent. A separate ROS job builds inside a `ros:humble` container, resolves dependencies with `rosdep`, and runs `colcon build` and `colcon test` across all four packages, so the ROS 2 wrapping is tested against a real distribution.

B. Synthetic Evaluation

Beyond per-component assertions, three closed-loop experiments drive the actual estimator cores—compiled directly from `prism_loc_core` and `prism_loc_fusion` sources with `g++` and Eigen, no ROS and no CMake—through end-to-end localization scenarios. All data are synthetic: worlds, scans, and inertial streams are generated by the experiment drivers themselves, with all randomness derived from fixed compile-time seeds, so every number and figure below is exactly reproducible from the drivers and CSV logs shipped in the repository (`experiments/`). We stress that this is a correctness and behavior demonstration on synthetic data, not a field-accuracy benchmark: no real

robot or sensor is involved, and the sensor-noise models match the assumptions of the filters by construction. A quantitative field evaluation on public datasets remains future work (Section VII).

MCL pose tracking and KLD adaptation. The first experiment exercises the laser2d pipeline exactly as localization `node.cpp` wires it. A $12\text{ m} \times 10\text{ m}$ occupancy grid (0.1 m/cell, the integration test’s room-border convention plus two interior pillars that break pose symmetry) hosts a simulated robot driven around a closed oval loop with semi-axes 3.6 m and 2.6 m ($7.2\text{ m} \times 5.2\text{ m}$ extent) for 65 s at 10 Hz (650 steps). At each step a 180-beam scan is ray-cast at the true pose, and a drifting odometry frame is synthesized by integrating noise-corrupted motion increments under the laser2d.yaml odometry model ($\alpha_{1..4} = 0.2$). The real ParticleFilter with the Laser2DLikelihoodField model and KLD-adaptive resampling—every parameter taken from laser2d.yaml—is seeded with a broad Gaussian prior ($\sigma = 1\text{ m}$, 0.4 rad) centered at the true start pose and updated in the node’s predict–correct–resample order, gated by `update_min_d = 0.2 m` and `update_min_a = 0.2 rad`. Three fixed seeds (101, 202, 303) are run. Fig. 2 shows the resulting position and yaw error: after convergence ($t \geq 10\text{ s}$, across all three seeds) the mean position error is 0.105 m (max 0.29 m) and the mean yaw error is 0.032 rad = 1.8° (max 0.18 rad). Fig. 3 shows the KLD-adaptive particle count over the same runs: the set starts at the 2000-particle ceiling under the broad prior, collapses to the 500-particle floor after the first post-convergence update, and holds there for the rest of the loop—a $4\times$ reduction in per-update likelihood evaluations once the posterior is concentrated.

Global relocation by branch-and-bound. The second experiment evaluates the BBS matcher (Section IV-B) as a kidnapped-robot relocalizer. On the asymmetric $6\text{ m} \times 6\text{ m}$ room from the BBS unit test (two interior wall stubs break the four-fold symmetry, so a single 360° scan pins the global pose uniquely), $N = 100$ true poses are drawn uniformly over free space with uniform yaw (fixed seed 20240607), and one 180-beam scan is ray-cast per pose. To span an easy-to-hard range, each scan is corrupted with synthetic clutter: a per-query clutter fraction drawn uniformly in $[0, 0.8]$ replaces each beam, with that probability, by a uniform-random spurious range—modeling people, glass, and objects absent from the static map. The real BranchAndBoundMatcher then searches the whole map with the laser2d.yaml `bbs_*` defaults. Defining success as position error $< 0.5\text{ m}$ and yaw error $< 10^\circ$, 97 of 100 queries succeed (97.0%); over all 100 queries the recovered pose has median position error 0.037 m (95th percentile 0.072 m) and median yaw error 0.48° (95th percentile 1.74°). Fig. 4 plots position error against the BBS score fraction (raw score over used beams): the acceptance threshold `bbs_min_score_fraction = 0.40` yields zero false accepts—all 3 failures score in 0.21–0.27 with $> 3\text{ m}$ and roughly 90° – 180° error, while 78

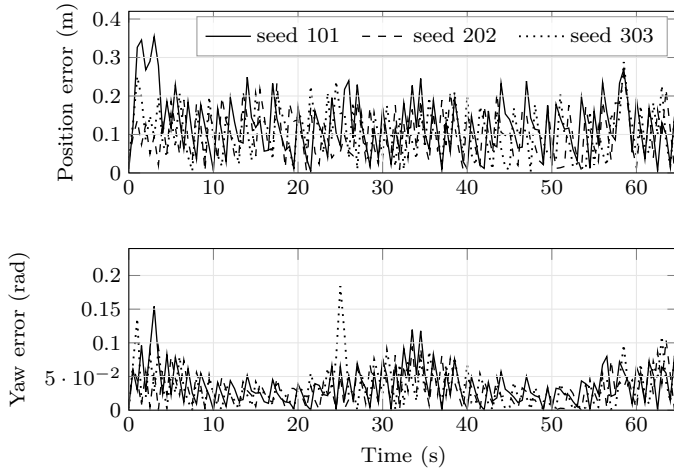


Fig. 2. Monte-Carlo-localization (MCL) tracking accuracy of the laser2d core on a synthetic occupancy grid. A simulated robot is driven around a closed oval loop with semi-axes 3.6 m and 2.6 m (7.2 m $\times$ 5.2 m extent) for 65 s at 10 Hz through a 12 m $\times$ 10 m room (two interior pillars break pose symmetry); 180-beam scans are generated by grid-march ray casting (test_helpers.hpp), and the odometry is a drifting frame produced by integrating noise-corrupted motion increments under the laser2d.yaml α model. Every plotted value comes from the CSV written by experiments/mcl_tracking.cpp, which runs the real prism_loc_core::ParticleFilter with the Laser2DLikelihoodField model and KLD resampling, wired (predict $\rightarrow$ correct $\rightarrow$ resample, gated by update_min_d/update_min_a) exactly as localization_node.cpp. Three fixed seeds are shown as thin lines. Starting from a broad 1 m prior the filter converges within a few updates and then tracks the ground truth for the whole loop: across the three seeds, steady-state ($t \geq 10$ s) mean position error is 0.105 m (max 0.29 m) and mean yaw error is 0.032 rad (1.8 $^\circ$, max 0.18 rad).

matches are accepted (all correct) and 19 correct fixes are conservatively rejected. The median score fraction is 0.632; successes span 0.22–0.97 while all failures fall below 0.28. Every query scoring at or above the 0.40 acceptance threshold is correct, so the threshold separates accepted matches from failures with a wide margin, even though three low-scoring successes overlap the failure score band. Median full-map query latency is 37.65 ms (min 8.77 ms, 95th percentile 106.06 ms; single-threaded, machine-dependent wall-clock; all accuracy numbers are deterministic across runs).

ESKF fusion under aiding dropout. The third experiment drives the actual prism_loc_fusion ESKF over a synthetic 60 s 3-D trajectory: a horizontal figure-eight ($x = 8 \sin \omega t$, $y = 4 \sin 2\omega t$, $\omega = 2\pi/30$) with a gentle ± 1.5 m altitude oscillation. The ideal body-frame specific force and angular rate computed from this analytic trajectory are corrupted with exactly the white-noise densities and constant biases the ESKF process model assumes (fusion3d.yaml defaults), plus fixed true biases the filter must estimate from zero; all noise derives from a single fixed seed (20240517). The IMU runs at 200 Hz, a simulated NDT source delivers 6-DoF pose updates at 10 Hz, and a simulated GNSS source delivers position

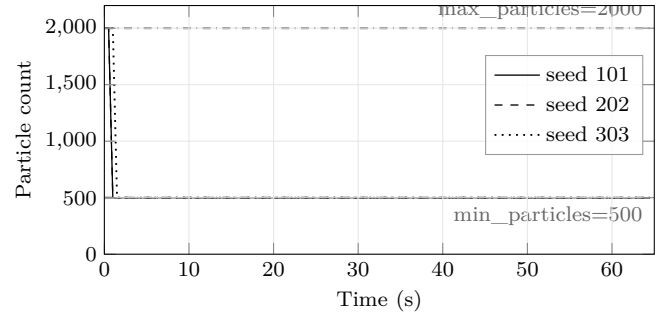


Fig. 3. KLD-adaptive resampling sizes the particle set to the current uncertainty (synthetic data; same run as Fig. 2, values from the CSV written by experiments/mcl_tracking.cpp). The dash-dotted lines mark the laser2d.yaml bounds min_particles=500 and max_particles=2000. Seeded with a broad prior, the filter starts at the 2000-particle ceiling; once the first laser corrections concentrate the posterior the KLD bound (Fox) shrinks the set to the 500-particle floor, where it stays for the remainder of the closed-loop run across all three fixed seeds. Steady-state tracking therefore costs 4 $\times$ fewer likelihood evaluations per update than the fixed worst-case ceiling, with no loss of the accuracy shown in Fig. 2.

updates at 1 Hz, with injected measurement noise matched to the filter’s R matrices. A 10 s window ($t \in [25, 35]$ s) withholds both aiding streams to expose IMU-only dead reckoning and recovery. Fig. 5 shows the position-error norm against the filter’s own 3σ position bound: windowed position RMSE is 0.016 m before, 0.125 m during, and 0.009 m after the dropout, with a peak error of 0.256 m at $t = 34.5$ s (end of the IMU-only blackout) recovering to 0.015 m within one aiding cycle by $t = 35$ s. The 3σ bound inflates from roughly 0.03 m under full aiding to a peak of 0.663 m at $t = 34.5$ s and stays above the true error throughout—the covariance the filter reports is consistent with its actual error.

Scope. This validation establishes that the estimators are individually correct, that all three sit behind one working ROS 2 contract, and that the assembled pipelines behave as intended in closed loop on synthetic data; it is deliberately not an accuracy benchmark. The planned quantitative field evaluation is discussed with the other roadmap items in Section VII.

VII. Limitations and Future Work

prism_loc is an architecture paper, and its scope is deliberately bounded. We state the current boundaries plainly so that they read as a roadmap rather than as hidden assumptions.

Planar state in the MCL backends. Both particle-filter backends estimate a planar pose (x, y, θ): the shared core represents each particle as an $SE(2)$ element, and the ndt3d backend evaluates its per-particle likelihood against a full 3D Gaussian-voxel map from that planar pose at a fixed base height. This suits wheeled robots on locally flat floors but does not track roll, pitch, or vertical motion. Full $SE(3)$ state exists only in the fusion3d backend, whose 15-dimensional error state carries position, velocity,

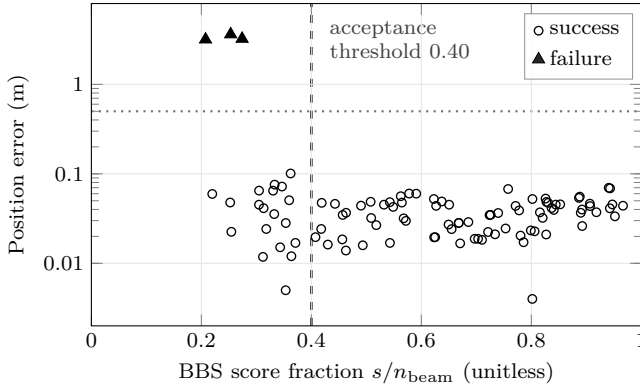


Fig. 4. Branch-and-bound (BBS) global relocalization for a kidnapped robot on a synthetic occupancy grid. From an asymmetric 6 m $\times$ 6 m room (two interior wall stubs break the four-fold symmetry, the same world as the Steps unit test), $N = 100$ true poses are drawn uniformly over free space with uniform yaw (fixed seed). For each pose a single 180-beam scan is produced by grid-march ray casting (test_helpers.hpp) and corrupted with synthetic clutter—each beam is, with a per-query probability drawn uniformly in $[0, 0.8]$, replaced by a spurious range—to span an easy-to-hard difficulty range. The real prism_loc_core::BranchAndBoundMatcher (bbs.cpp) then searches the whole map with the laser2d.yaml bbs_* defaults (± 10 m linear, $\pm \pi$ yaw at 1° steps, max_depth = 6, max_beams = 120). Each marker is one query: position error versus BBS score fraction (raw score over used beams); the dashed line is the acceptance threshold bbs_min_score_fraction = 0.40. A query counts as a success when position error < 0.5 m and yaw error $< 10^\circ$. The score cleanly separates outcomes: all three failures score below 0.40 (with > 3 m error), and every match at or above the threshold is correct, so the threshold yields zero false accepts over the run (the 19 correct fixes it rejects are its conservative margin). Success rate is $97/100 = 97\%$; median full-map query latency is 37.7 ms (single-threaded, machine-dependent); median position error 0.037 m and median yaw error 0.48° .

orientation, and IMU biases. Extending the MCL core to a 6-DoF particle state, or coupling the planar filter to an external attitude source, is future work.

2D-only global relocalization. The branch-and-bound search used for kidnapped-robot recovery (Section IV-B) operates on a rasterized 2D occupancy pyramid and returns an $SE(2)$ pose. There is no analogous 3D global localization for the ndt3d or fusion3d backends; those currently rely on an operator-supplied initial pose (via /initialpose) and local tracking. A 3D branch-and-bound or learned place-recognition front end is on the roadmap.

No online sensor time-offset estimation. The fusion3d ESKF propagates on IMU samples and corrects with LiDAR-NDT and GNSS measurements at their stamped times, but it does not estimate the temporal offset between the LiDAR/GNSS clocks and the IMU clock, nor does it self-calibrate the LiDAR-IMU extrinsic; both are treated as known inputs. Jointly estimating time offset and extrinsics would improve robustness under poorly synchronized hardware.

Loosely-coupled LiDAR update. In fusion3d, PCL NDT registers each live cloud to the prior map to produce a 6-DoF pose measurement, which the ESKF then fuses as a loosely-coupled update. This keeps the ROS-free filter

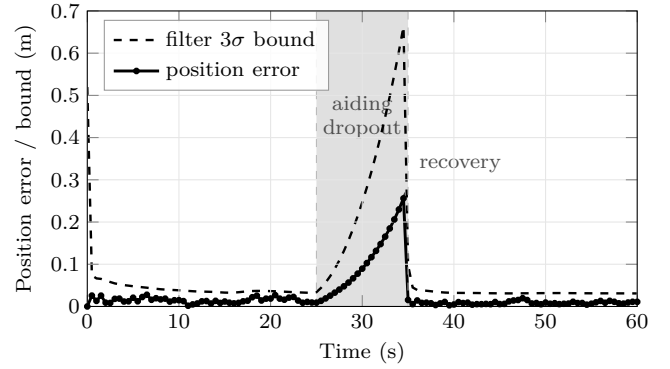


Fig. 5. Error-state Kalman filter (ESKF) fusion of a synthetic 200 Hz IMU with 10 Hz 6-DoF pose (NDT-like) and 1 Hz position (GNSS-like) updates over a smooth 60 s 3-D figure-eight trajectory. All data are synthetic: the IMU is generated analytically from the trajectory and corrupted with the white-noise densities and constant biases the ESKF process model assumes (fusion3d.yaml defaults), and every plotted value comes from the CSV written by experiments/eskf_fusion.cpp, which runs the real prism_loc_fusion::Eskf. Both aiding streams are withheld for $t \in [25, 35]$ s (shaded): the position error grows monotonically to 0.26 m on IMU dead-reckoning alone while the filter’s own 3σ bound inflates to 0.66 m, and both collapse within one aiding cycle once updates resume. Windowed RMSE (from the CSV): 0.016 m before, 0.125 m during, and 0.009 m after the dropout.

core independent of the registration engine, but it forgoes the accuracy of tightly-coupled formulations that fold raw scan residuals directly into the filter Jacobian, as in FAST-LIO [11]. The loose coupling is an architectural choice favoring modularity and testability over last-percent accuracy.

Single static map. All three backends assume one prior map of a static world, loaded once at startup; they neither update the map online nor reason about moving obstacles or long-term scene change. Map maintenance and change detection are out of scope for the present system.

No field-accuracy evaluation yet. The validation reported in Section VI is deterministic, ROS-free unit and integration testing plus continuous integration, complemented by a seeded synthetic evaluation; it establishes that each estimator core behaves as specified, including in closed loop, but the synthetic worlds and sensor-noise models are generated to match the filters’ assumptions, so it is not a quantitative accuracy study. We have not yet measured trajectory error on physical or public datasets. Planned work is an evaluation reporting absolute and relative trajectory error (ATE/RPE) per the TUM RGB-D conventions [15] on public datasets, applied to the per-backend map $\rightarrow$ odom estimates. Until then we make no accuracy claims.

VIII. Conclusion

We presented prism_loc, a LiDAR localization stack whose contribution is architectural rather than algorithmic: three known estimator families — likelihood-field MCL, NDT-MCL, and an error-state Kalman filter that

fuses LiDAR, IMU, and RTK-GNSS — are placed behind a single ROS 2 topic and TF contract, so that a 2D LiDAR, a 3D LiDAR, or a sensor-fused configuration is a deployment choice rather than a separate software stack. Each estimator is implemented as a pure C++ core with no dependence on the middleware runtime, making the filters unit-testable without ROS and letting the two MCL backends share one predict–weight–resample core that differs only in its measurement model. We claim no new filter theory; the novelty is the factoring that makes these families interchangeable and independently testable behind a familiar interface. `prism_loc` is released under the Apache-2.0 license at https://github.com/kjungmo/prism_loc.

Appendix Reproducibility

Every number and figure in Section VI-B regenerates from the repository alone; this appendix consolidates the recipe.

Environment. The experiment drivers and the core test suites need only a C++17 compiler and the Eigen 3 headers (Debian/Ubuntu packages `g++` and `libeigen3-dev`); the unit tests of Section VI-A additionally use GoogleTest (`libgtest-dev`). ROS 2 Humble is required only for the two node packages; continuous integration builds those in a `ros:humble` container (`.github/workflows/ci.yml`).

Building and running the drivers. The three drivers compile the estimator cores directly from source—no CMake, no ROS. From the repository root:

```
g++ -O2 -std=c++17 -I prism_loc_core/include \
-I prism_loc_core/test \
-I /usr/include/eigen3 \
prism_loc_core/src/*.cpp \
experiments/mcl_tracking.cpp -o /tmp/exp_mcl
/tmp/exp_mcl

g++ -O2 -std=c++17 -I prism_loc_core/include \
-I prism_loc_core/test \
-I /usr/include/eigen3 \
prism_loc_core/src/*.cpp \
experiments/bbs_relocalization.cpp \
-o /tmp/exp_bbs
/tmp/exp_bbs

g++ -O2 -std=c++17 \
-I prism_loc_fusion/include \
-I /usr/include/eigen3 \
prism_loc_fusion/src/*.cpp \
experiments/eskf_fusion.cpp -o /tmp/exp_eskf
/tmp/exp_eskf > docs/paper/data/eskf_track.csv
```

All randomness derives from fixed compile-time seeds (Table II), so every CSV column is bit-for-bit reproducible, with the single exception of the wall-clock `time_ms` latency column of the BBS experiment, which is machine-dependent.

Parameters. The drivers mirror the shipped defaults. Every ROS 2 parameter and its in-code default is documented in `PARAMS.md` at the repository root; the deployed overrides are

TABLE II
Experiment drivers, fixed seeds, and committed artifacts. Drivers live under `experiments/`; CSVs under `docs/paper/data/`.

Driver (.cpp)	Seed(s)	Figure(s)	CSV(s)
<code>mcl_tracking</code>	101, 202, 303	2, 3	<code>mcl_tracking</code> , <code>mcl_particles</code>
<code>bbs_relocalization</code>	20240607	4	<code>bbs_relocalization</code>
<code>eskf_fusion</code>	20240517	5	<code>eskf_track</code>

the YAML files `prism_loc/params/laser2d.yaml`, `prism_loc/params/ndt3d.yaml`, and `prism_loc_fusion_ros/params/fusion3d.yaml`.

Number tracing. The committed CSVs under `docs/paper/data/` are the artifacts of record. The guard script `docs/paper/check_numbers.py` recomputes every headline statistic quoted in Section VI-B from those CSVs and exits nonzero if any quoted number no longer matches, so the paper’s statistics cannot silently drift from the data.

References

- [1] T. Moore and D. Stouch, “A generalized extended Kalman filter implementation for the robot operating system,” in *Intelligent Autonomous Systems 13: Proceedings of the 13th International Conference (IAS-13)*, ser. *Advances in Intelligent Systems and Computing*, vol. 302. Cham: Springer, 2014, pp. 335–348.
- [2] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the Robot Operating System 2,” *Robotics and Autonomous Systems*, vol. 168, p. 104493, 2023.
- [3] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A navigation system,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 2718–2725.
- [4] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, “Monte Carlo localization for mobile robots,” in *Proceedings of the 1999 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2. Detroit, MI, USA: IEEE, May 1999, pp. 1322–1328.
- [5] D. Fox, W. Burgard, F. Dellaert, and S. Thrun, “Monte Carlo localization: Efficient position estimation for mobile robots,” in *Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI/IAAI)*. Orlando, FL, USA: AAAI Press / The MIT Press, 1999, pp. 343–349.
- [6] S. Thrun, D. Fox, W. Burgard, and F. Dellaert, “Robust Monte Carlo localization for mobile robots,” *Artificial Intelligence*, vol. 128, no. 1-2, pp. 99–141, May 2001.
- [7] P. Biber and W. Straßer, “The normal distributions transform: A new approach to laser scan matching,” in *Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3. Las Vegas, NV, USA: IEEE, Oct. 2003, pp. 2743–2748.
- [8] M. Magnusson, A. Lilienthal, and T. Duckett, “Scan registration for autonomous mining vehicles using 3D-NDT,” *Journal of Field Robotics*, vol. 24, no. 10, pp. 803–827, 2007.
- [9] J. Solà, “Quaternion kinematics for the error-state Kalman filter,” Nov. 2017, arXiv:1711.02508 [cs.RO], tutorial/technical report. [Online]. Available: <https://arxiv.org/abs/1711.02508>
- [10] G. Wan, X. Yang, R. Cai, H. Li, Y. Zhou, H. Wang, and S. Song, “Robust and precise vehicle localization based on multi-sensor fusion in diverse city scenes,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. Brisbane, QLD, Australia: IEEE, May 2018, pp. 4670–4677.
- [11] W. Xu and F. Zhang, “FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated Kalman filter,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, Apr. 2021.

- [12] K. Koide, J. Miura, and E. Menegatti, "A portable three-dimensional LiDAR-based system for long-term and wide-area people behavior measurement," *International Journal of Advanced Robotic Systems*, vol. 16, no. 2, Mar. 2019.
- [13] E. B. Olson, "Real-time correlative scan matching," in 2009 IEEE International Conference on Robotics and Automation (ICRA). Kobe, Japan: IEEE, 2009, pp. 4387–4393.
- [14] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," in 2016 IEEE International Conference on Robotics and Automation (ICRA). Stockholm, Sweden: IEEE, 2016, pp. 1271–1278.
- [15] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Vilamoura, Portugal: IEEE, Oct. 2012, pp. 573–580.
- [16] D. Fox, "Adapting the sample size in particle filters through KLD-sampling," *The International Journal of Robotics Research*, vol. 22, no. 12, pp. 985–1003, 2003.